

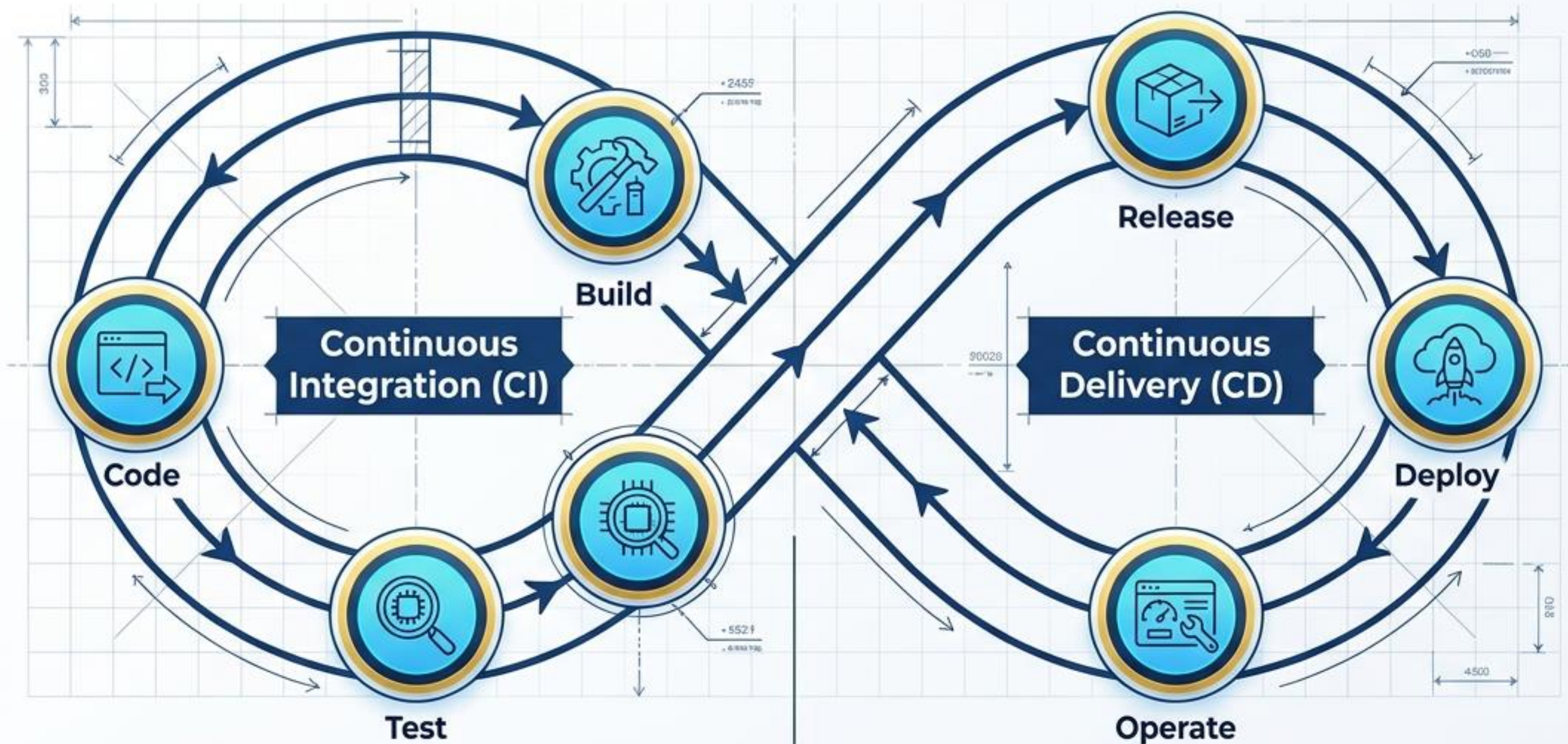
Creating a **CI/CD** **Pipeline** on **Azure**

The Blueprint for
Continuous Integration
& **Deployment**

Week 5 Practical | **Azure DevOps**
& **GitHub** Integration

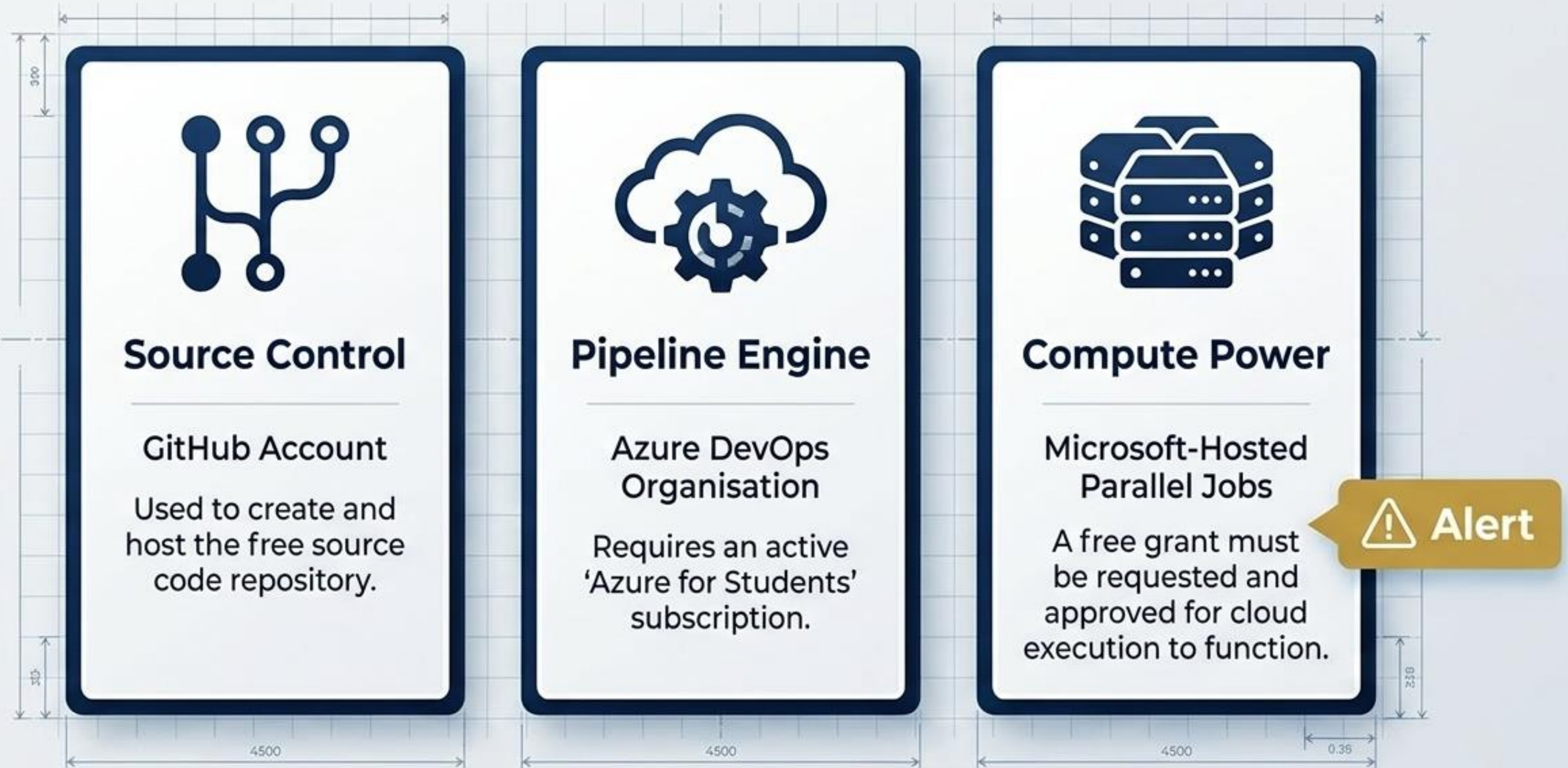


Decoding the CI/CD Engine

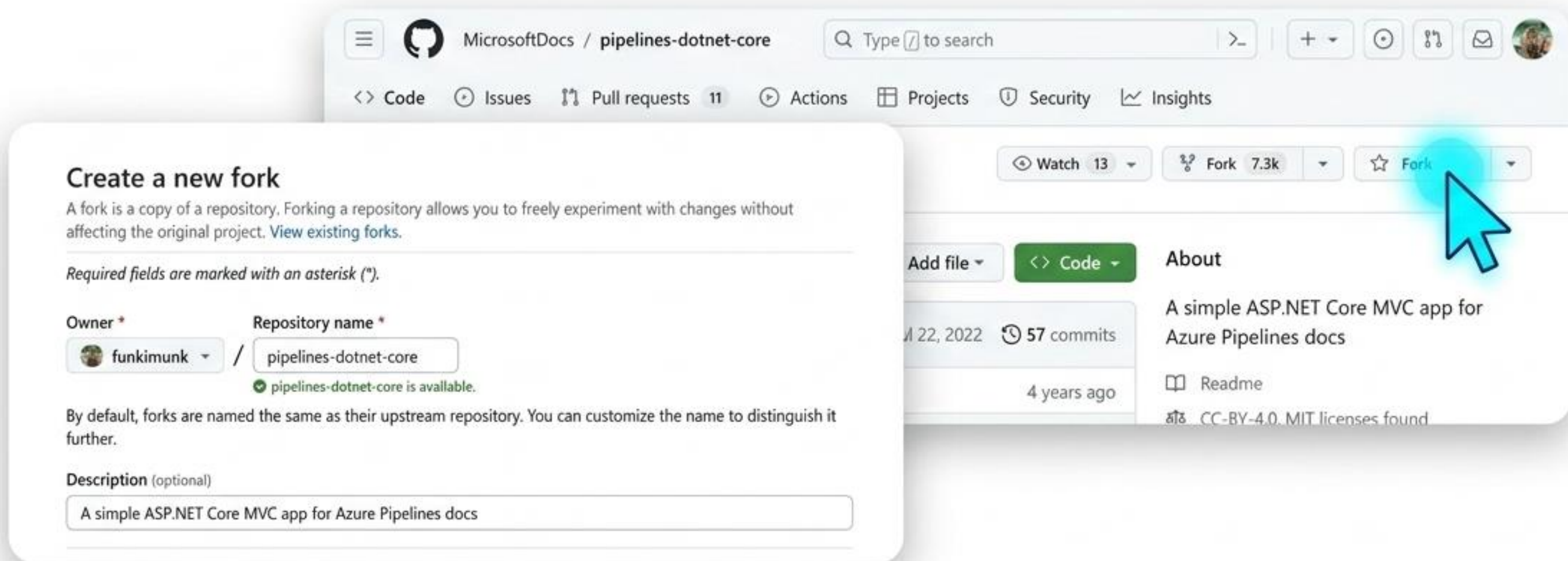


Azure Pipelines automates this entire lifecycle. It transforms manual code updates on GitHub into automated, cloud-hosted build-deploy-test workflows, enabling fast and scalable software routines.

Project Prerequisites: The Required Arsenal



Phase 1: Forking the Source Control



1

Navigate to the target repository (MicrosoftDocs/pipelines-dotnet-core).

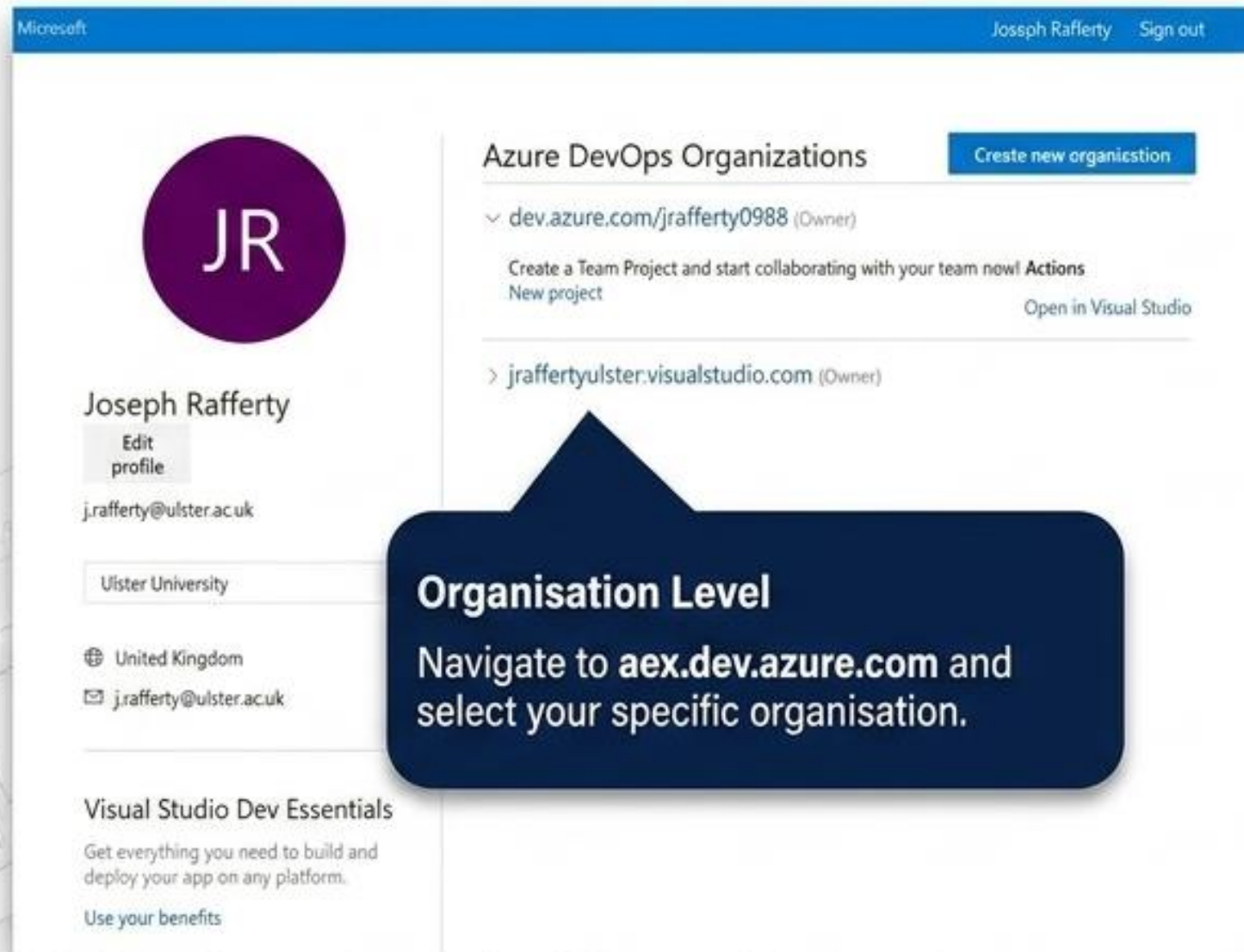
2

Click **Fork** to clone the remote codebase directly into your personal GitHub account.

3

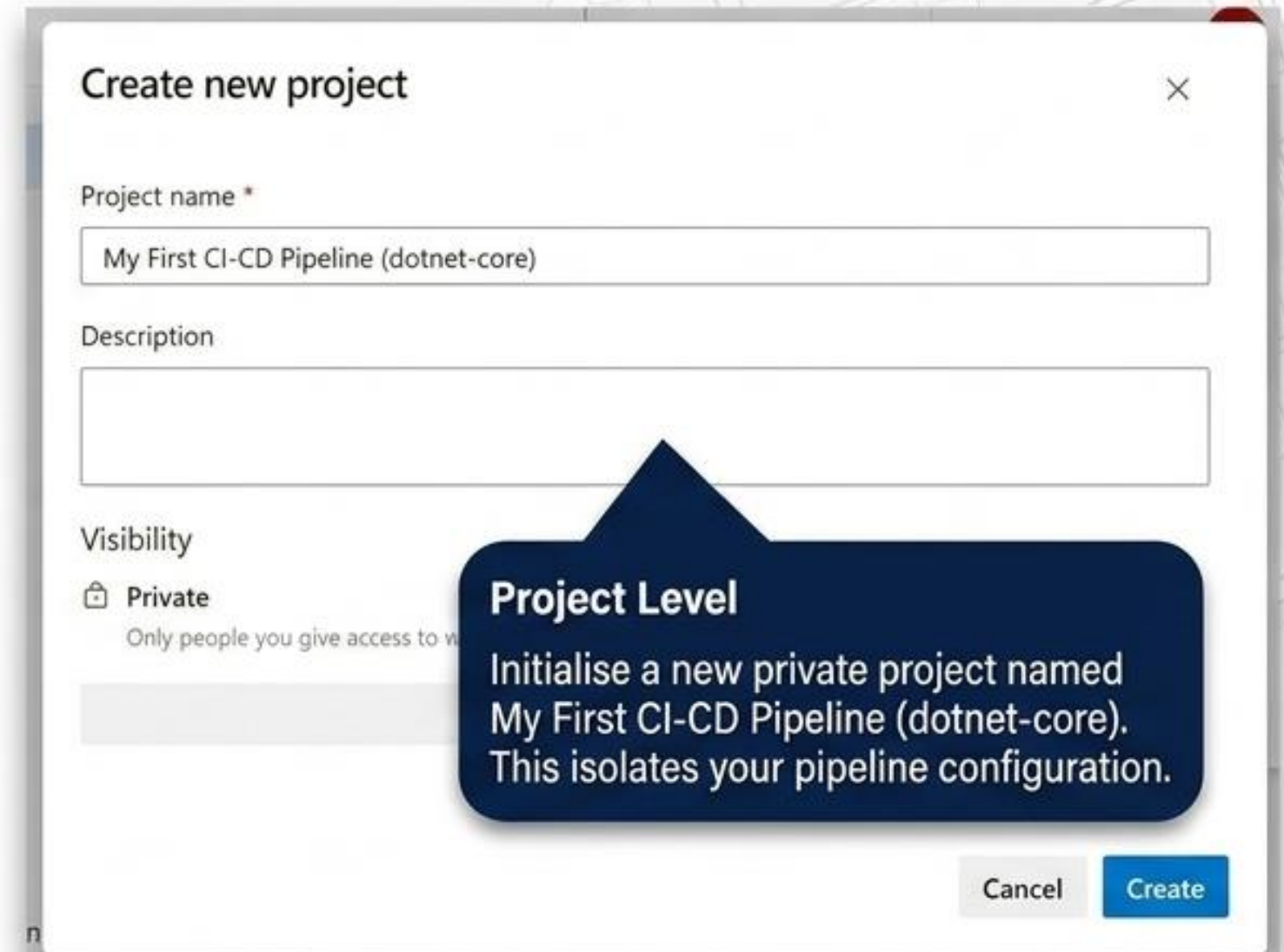
Confirm yourself as the **Owner** and retain the default repository name to prevent conflicts.

Phase 2: Initialising the DevOps Sandbox



The screenshot shows the Azure DevOps Organizations page for user Joseph Rafferty. The page has a blue header with the Microsoft logo and the user's name and sign-out link. On the left, there's a profile section with a circular avatar containing 'JR', the name 'Joseph Rafferty', an 'Edit profile' button, and contact information: 'j.rafferty@ulster.ac.uk' and 'Ulster University'. The main section is titled 'Azure DevOps Organizations' and includes a 'Create new organisation' button. Below this, there are two listed organizations: 'dev.azure.com/jrafferty0988 (Owner)' and 'jraffertyulster.visualstudio.com (Owner)'. A callout box points to the second organization.

Organisation Level
Navigate to aex.dev.azure.com and select your specific organisation.



The screenshot shows the 'Create new project' dialog box. It has a title bar with a close button. The form includes a 'Project name' field with the text 'My First CI-CD Pipeline (dotnet-core)', a 'Description' field, and a 'Visibility' section with a 'Private' option selected. A callout box points to the project name field. At the bottom right, there are 'Cancel' and 'Create' buttons.

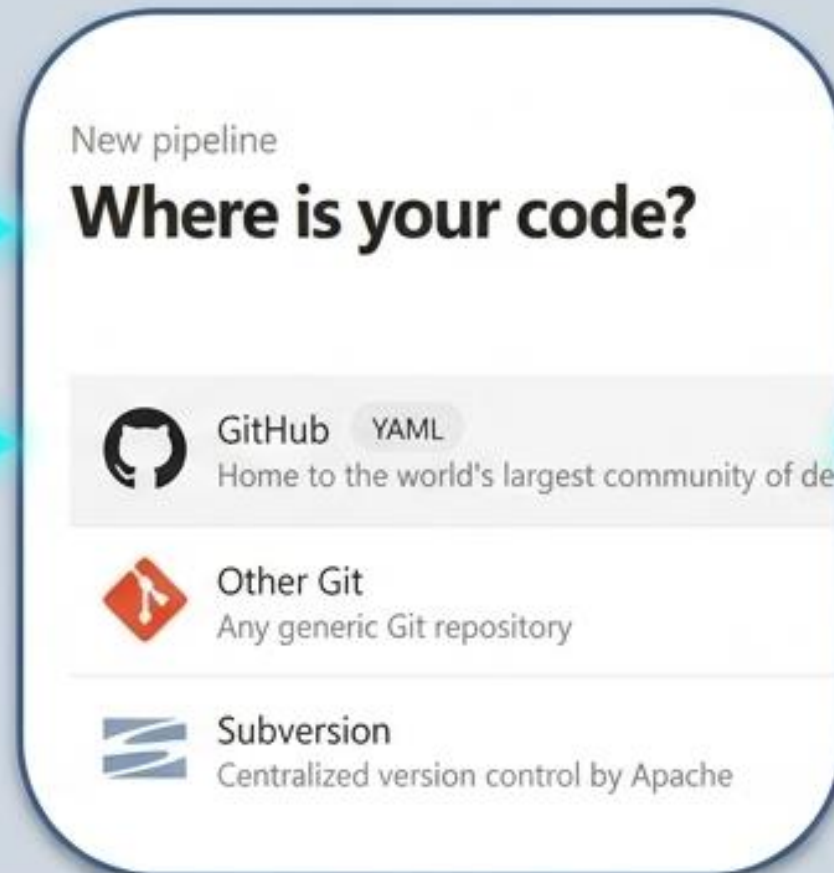
Project Level
Initialise a new private project named My First CI-CD Pipeline (dotnet-core). This isolates your pipeline configuration.

Phase 3: Building the Integration Bridge (OAuth)

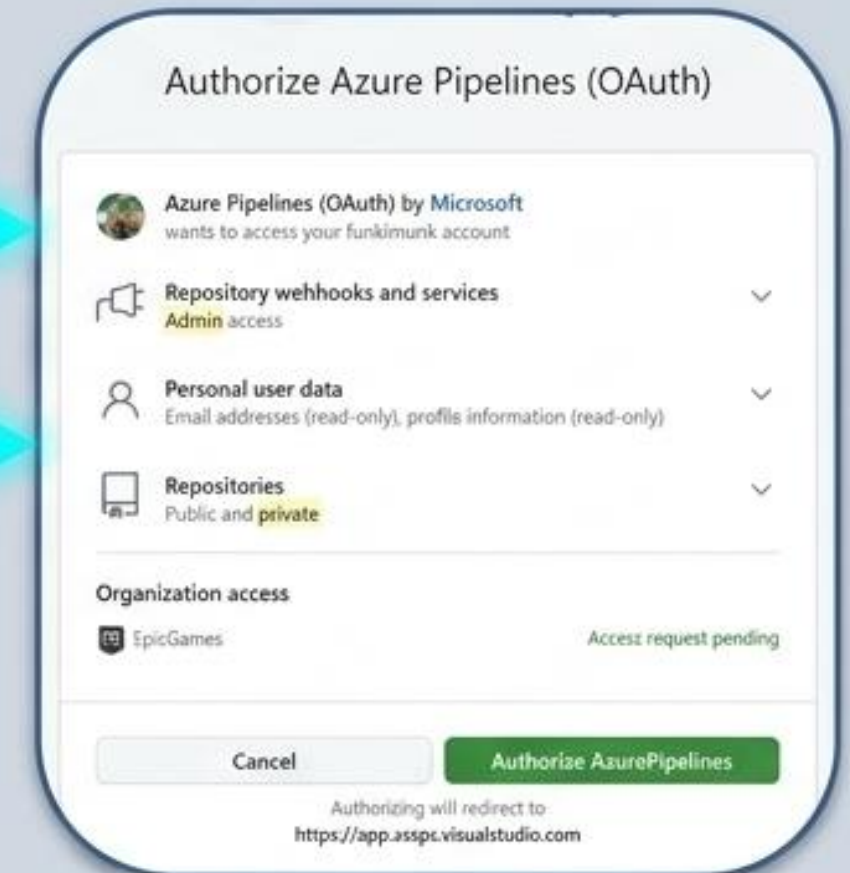
Step 1



Step 2

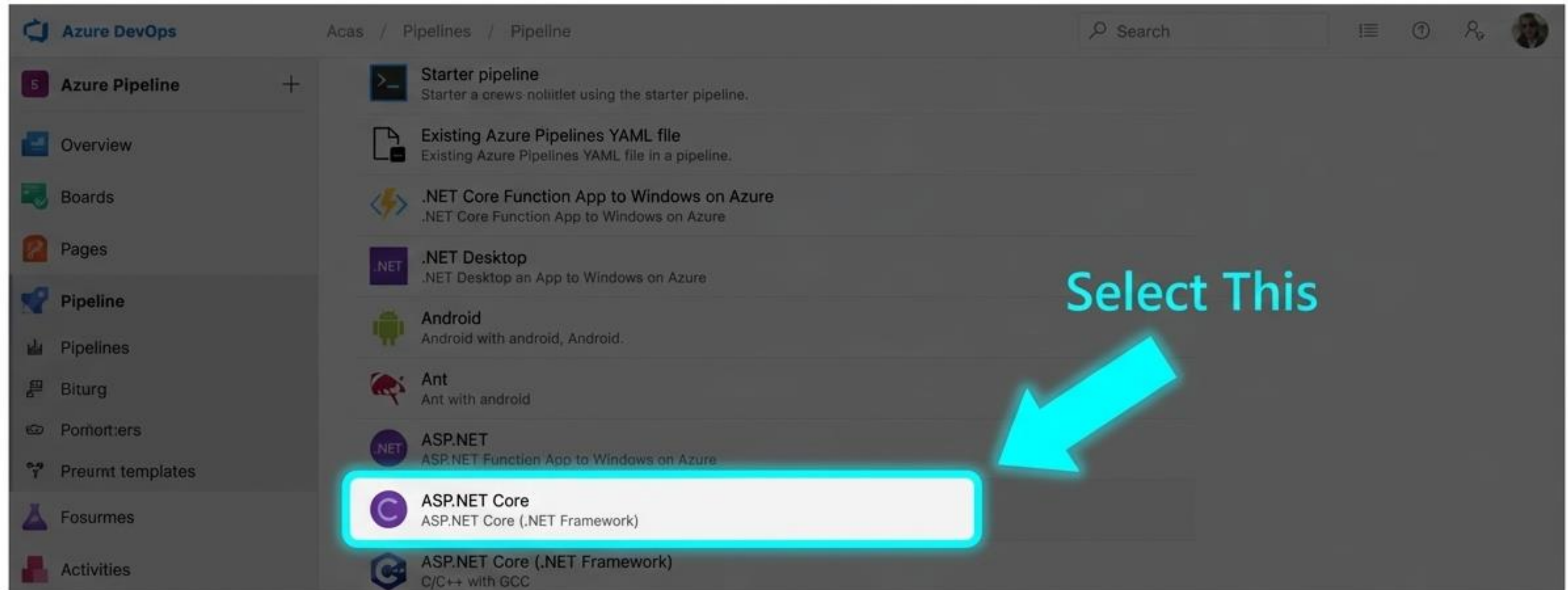


Step 3



Azure DevOps requires explicit read/write access to your GitHub repository metadata. Approving this OAuth handshake allows Azure to detect code commits automatically and trigger the build pipeline without manual intervention.

Phase 4: Selecting the Pipeline Blueprint



Note:

The forked GitHub repository contains an ASP.NET Core MVC application. You must select the matching ASP.NET Core YAML template to ensure Azure provisions the correct .NET build environment for compilation.

Infrastructure as Code: Dissecting the YAML

```
1  # ASP.NET Core
2  # Build and test ASP.NET Core projects targeting .NET Core.
3  # Add steps that run tests, create a NuGet package, deploy, and more:
4  # https://docs.microsoft.com/azure/devops/pipelines/languages/dotnet-core
5
6  trigger:
7    - master
8
9  pool:
10    vmImage: ubuntu-latest
11
12  variables:
13    buildConfiguration: 'Release'
14
15  steps:
16    - script: dotnet build --configuration $(buildConfiguration)
17      displayName: 'dotnet build $(buildConfiguration)'
18
```

The Event

Instructs Azure to run the pipeline automatically anytime new code is pushed to this branch.

The Environment

Provisions a fresh Ubuntu Linux virtual machine in the cloud to act as the build server.

The Action

The specific terminal command the agent executes to compile the application.

Phase 5: Execution & The Parallelism Roadblock



The Problem ❌

New Azure DevOps accounts restrict Microsoft-hosted agents to prevent abuse. If your pipeline fails with a “No hosted parallelism” error, the build cannot start.

We've received your request to increase free parallelism in Azure DevOps.

Please note that your request was **Completed**

Request Details:



Name	Joe Rafferty
Email	j.rafferty@ulster.ac.uk
Organization Name	jrafferty0988
Parallelism Type	Private

The Resolution ✅

A free parallelism grant must be requested via Microsoft.

Once the confirmation email arrives, the stalled pipeline can be rerun successfully.

Compute Alternatives: Cloud vs. Local Agents

	Microsoft-Hosted Agent (Cloud)	Local Agent (Self-Hosted)
Maintenance	Zero (Microsoft handles VM updates)	High (User manages software environment)
Access / Cost	Requires approved grant and wait time	Immediate access, bypasses grant delays
Performance	Standardised, predictable VM	Dependent on the hardware specifications of your PC

Takeaway: Establishing a local Windows agent is the recommended immediate workaround if grant approval is delayed.

Validating Pipeline Success

Status

The green checkmark confirms a successful compilation.

Trigger Metadata

Shows the exact commit message and the target branch.

The screenshot displays the Azure DevOps interface for a pipeline named "My First CI-CD Pipeline". The pipeline is in a successful state, indicated by a green checkmark. The interface shows the pipeline's summary, including the repository and version, the time started and elapsed, and the jobs. The jobs table shows a single job named "Job" with a status of "Success" and a duration of 23s. The interface also includes a sidebar with navigation options like Overview, Boards, Repos, Pipelines, Environments, Library, Test Plans, and Artifacts. The top navigation bar shows the pipeline's path and a search bar.

Summary Code Coverage

Manually run by Joseph R. Ferty

Repository and version
funkimunk/pipelines-dotnet-core
master 16312c13

Time started and elapsed
Just now
30s

Related
0 work items
0 artifacts

Tests and coverage
Get started

Jobs

Name	Status	Duration
Job	Success	23s

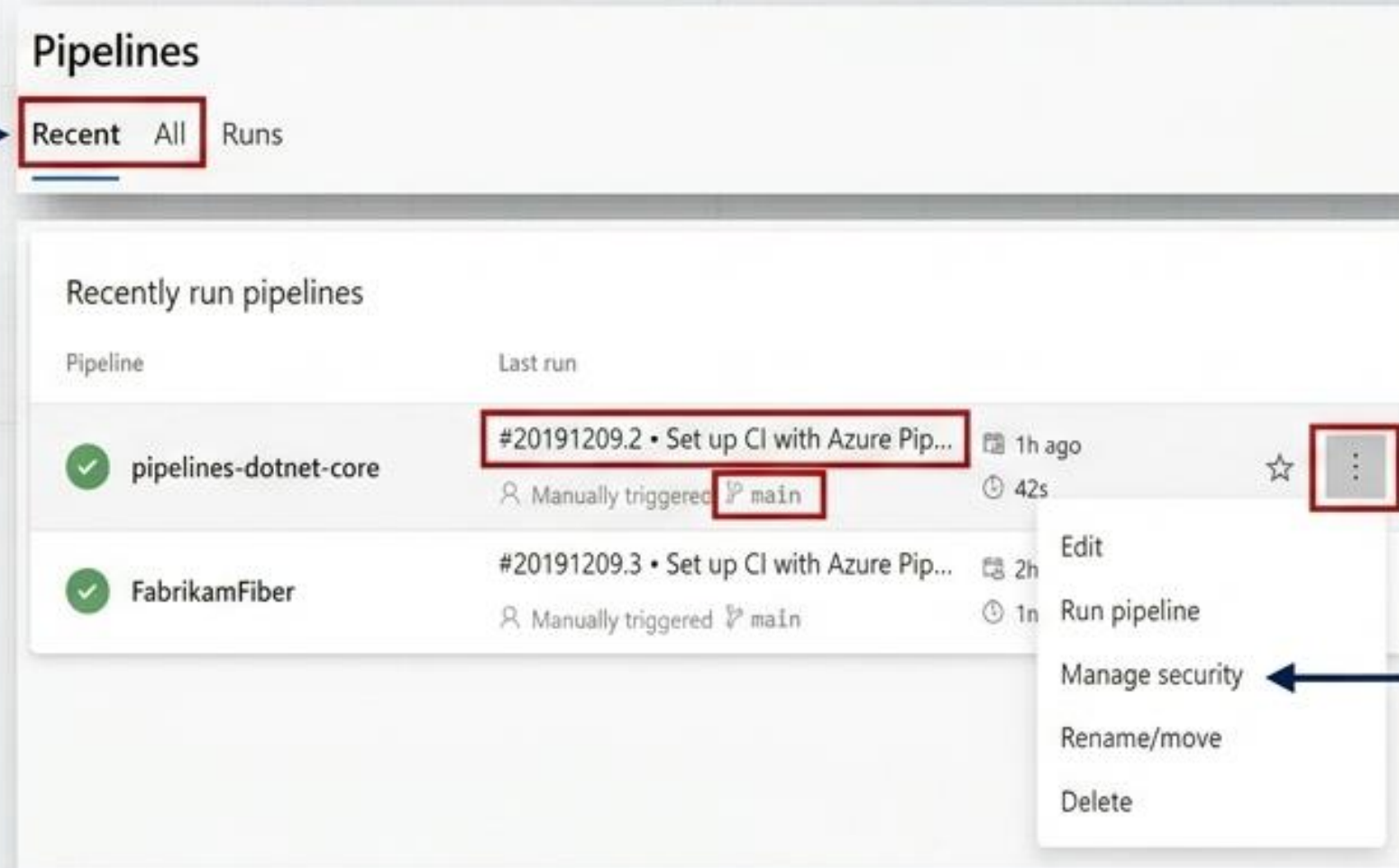
Performance

Displays the total duration metrics taken to provision the VM and build the code.

Managing Pipeline Health & History

Dashboard Navigation

Use the top tabs to toggle between recently run pipelines and the complete historical log of all individual runs.



The screenshot shows the 'Pipelines' dashboard with the 'Recent' tab selected. Below the tabs, there's a section titled 'Recently run pipelines'. It contains a table with columns 'Pipeline' and 'Last run'. The first row shows a pipeline named 'pipelines-dotnet-core' with a status icon (green checkmark) and a last run entry '#20191209.2 • Set up CI with Azure Pip...' triggered manually on the 'main' branch 1 hour ago, taking 42 seconds. A context menu is open for this pipeline, listing actions: Edit, Run pipeline, Manage security, Rename/move, and Delete. The 'Manage security' option is highlighted by an arrow from the 'Run Management' text block.

Pipeline	Last run
 pipelines-dotnet-core	#20191209.2 • Set up CI with Azure Pip... Manually triggered main 1h ago 42s
 FabrikamFiber	#20191209.3 • Set up CI with Azure Pip... Manually triggered main 2h 1m

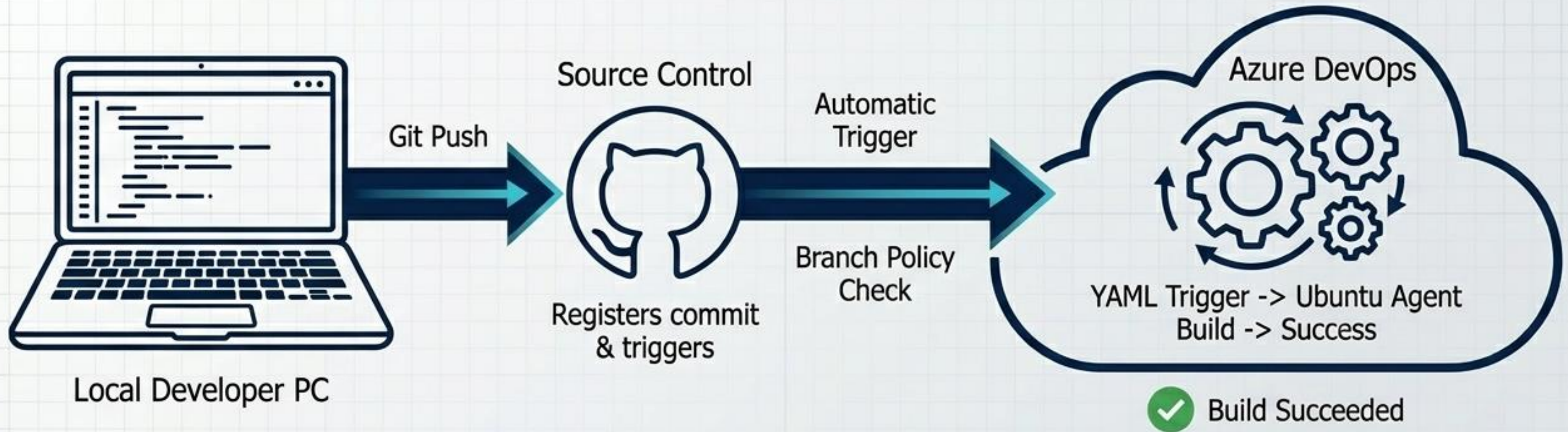
Run Management

The context menu (...) allows you to manually re-run pipelines, edit the YAML directly, or apply retention policies to lock specific historical builds from being auto-deleted.

Local to Cloud: Initialising Git via Terminal



The Complete Code-to-Cloud Workflow



The Big Picture: You have successfully built a living, automated engine. A single command executed in a local terminal now safely builds, tests, and validates your software in the cloud without further manual intervention.



Resource Management & Next Steps

Resource Clean Up

Always delete Azure App Services and associated resources from the DevOps Starter dashboard once testing is complete to prevent unwanted charges to your student account.

Further Learning

With Continuous Integration (CI) now successfully established, the infrastructure is ready for the next evolution: configuring deployment stages for Continuous Delivery (CD).